Domain Rules

RoomieMatch – Requirements Document

Members: Max Siebengartner, Zain Rizvi

Domain Rules

1. A User must register with a name, email, password, age, gender, occupation, bio, and an optional profile photo before accessing the platform.
2. Every User has one UserPreferences record that stores their lifestyle habits: cleanliness level, sleep schedule, pet friendliness, smoking preference, noise tolerance, guest policy, and work schedule.
3. A User may post one or more Listings. Each Listing includes a title, description, rent price, location (address, city, state, zip code), available date, number of rooms, number of bathrooms, and an active/inactive status.
4. A Listing may display zero or more Photos. Each Photo stores an image URL and the time it was uploaded.
5. A User may make a UserInterest to express interest in a Listing. Each UserInterest stores the renter, the listing, a status (e.g., Pending, Accepted, Rejected), and the date the interest was created.
6. A Listing can receive interest from zero or more Users, and a User can make interest in zero or more Listings.r
7. A User may send and receive Messages in the context of a specific Listing. Each Message stores the sender, receiver, associated listing, message content, sent timestamp, and a read status.
8. A User may create a Report against another User or against a Listing. Each Report records the reporter, the reported user or listing, a written reason, and the time it was created.
9. A User can create zero or more Reports, and a User or Listing can be the subject of zero or more Reports.
10. All User data is retained even if an account is deactivated, preserving associated messages, interests, and reports.

Nouns: User, UserPreferences, Listing, Photo, ListingPhoto, ProfilePhoto, UserInterest, Message, Report

Verbs:

- User **registers**
- User **has** UserPreferences
- User **posts** Listings
- Listing **displays** Photos
- User **makes** UserInterest (expresses interest in a Listing)
- User **sends/receives** Messages
- User **creates** Reports
- Data **is retained** on deactivation

Relational Schema

User(user_id, name, email, password_hash, age, gender, occupation, bio, profile_photo, created_at)

UserPreferences(preference_id, user_id, cleanliness_level, sleep_schedule, pet_friendly, smoking_allowed, noise_tolerance, guests_allowed, work_schedule)

Listing(listing_id, user_id, title, description, rent_price, location, city, state, zip_code, available_date, num_rooms, num_bathrooms, is_active, created_at)

ListingPhoto(photo_id, listing_id, data)

ProfilePhoto(photo_id, user_id, data)

UserInterest(interest_id, renter_id, listing_id, status, created_at)

Message(message_id, sender_id, receiver_id, listing_id, content, sent_at, read)

Report(report_id, reporter_id, reported_user_id, reported_listing_id, reason, created_at)

BCNF Proof

A relation is in BCNF if for every functional dependency X determines Y, X is a superkey. All relations in this schema have a single primary key that determines all other attributes, with no partial or transitive dependencies.

User: user_id determines name, email, password_hash, age, gender, occupation, bio, profile_photo, created_at. user_id is the primary key and superkey. In BCNF.

UserPreferences: preference_id determines user_id, cleanliness_level, sleep_schedule, pet_friendly, smoking_allowed, noise_tolerance, guests_allowed, work_schedule. user_id is also a superkey since each user has exactly one preferences record. Both determinants are superkeys. In BCNF.

Listing: listing_id determines user_id, title, description, rent_price, location, city, state, zip_code, available_date, num_rooms, num_bathrooms, is_active, created_at. listing_id is the primary key and superkey. In BCNF.

ListingPhoto: photo_id determines listing_id, data. photo_id is the primary key and superkey. In BCNF.

ProfilePhoto: photo_id determines user_id, data. photo_id is the primary key and superkey. In BCNF.

UserInterest: interest_id determines renter_id, listing_id, status, created_at. (renter_id, listing_id) is also a candidate key since a user can only express interest in a listing once. Both determinants are superkeys. In BCNF.

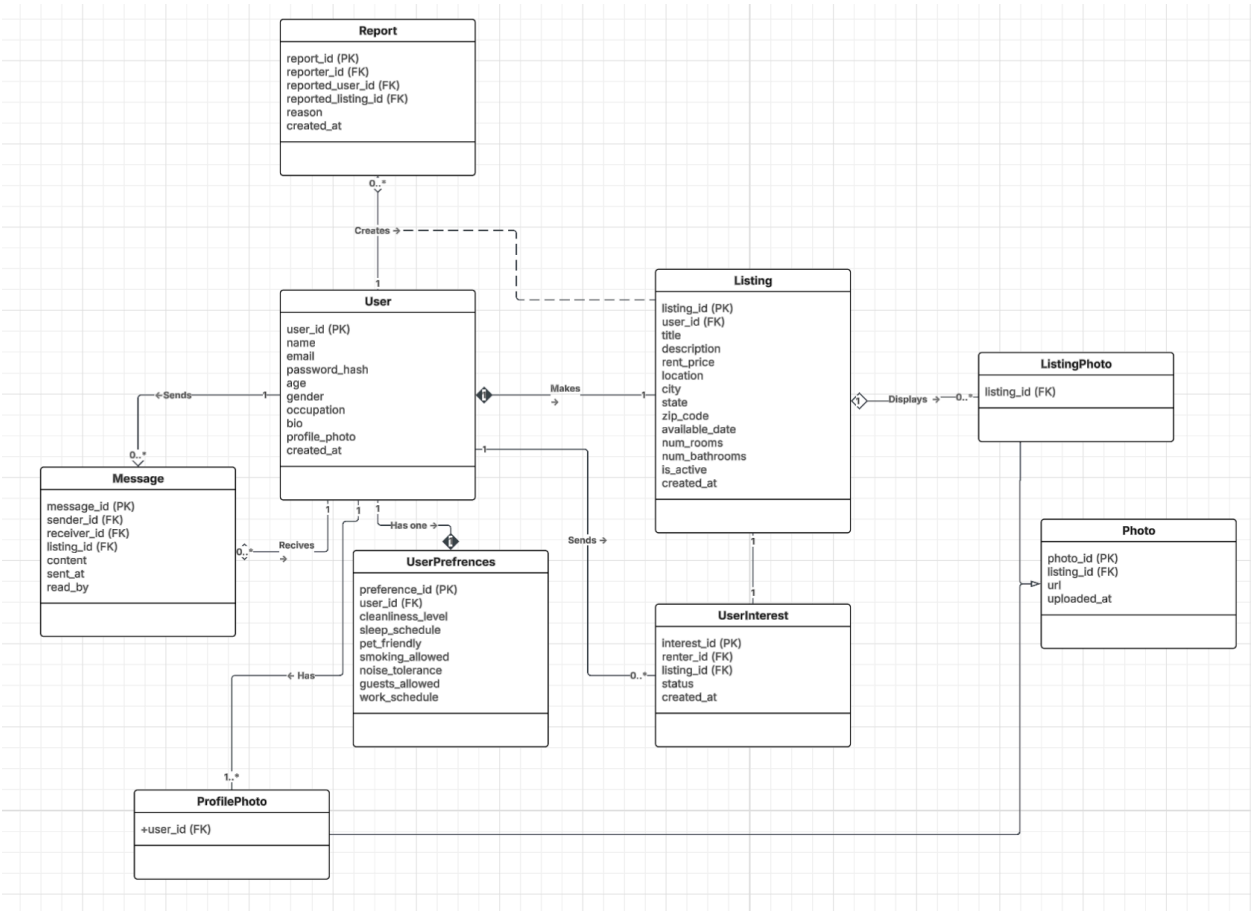
Message: message_id determines sender_id, receiver_id, listing_id, content, sent_at, read. message_id is the primary key and superkey. In BCNF.

Report: report_id determines reporter_id, reported_user_id, reported_listing_id, reason, created_at. report_id is the primary key and superkey. In BCNF.

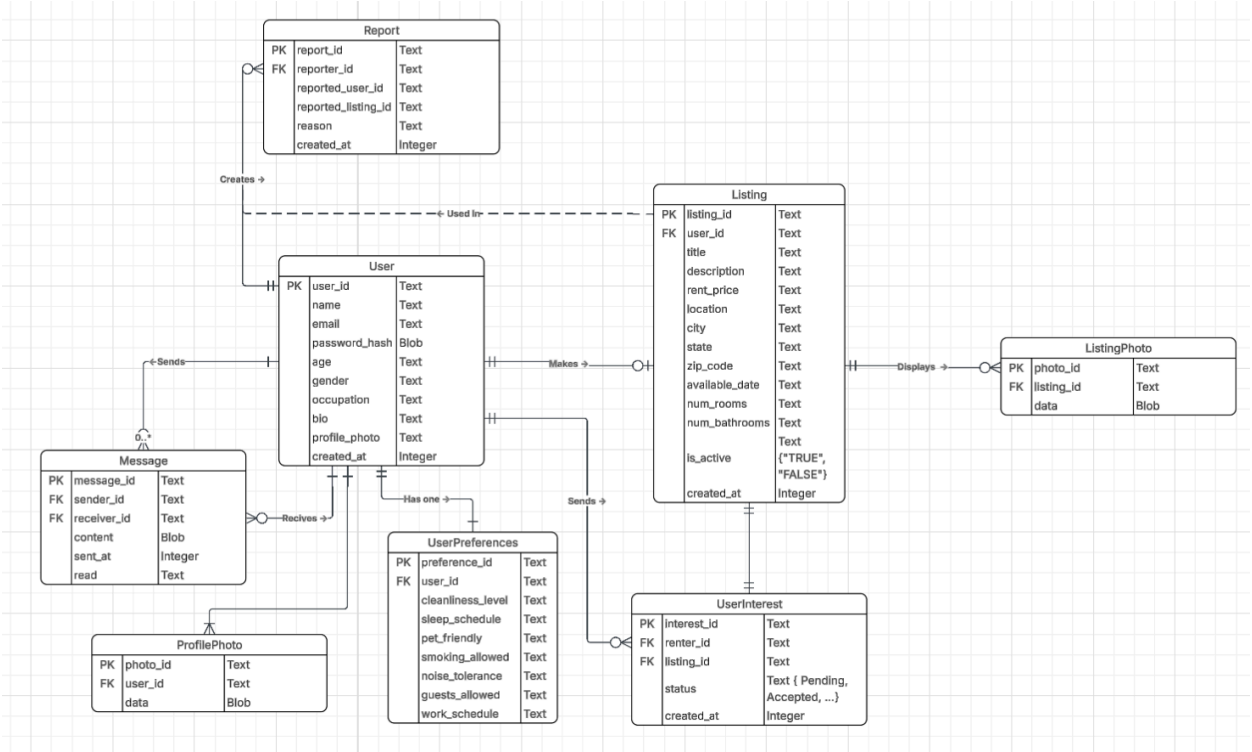
Diagrams

https://lucid.app/lucidchart/4b2d71d8-e729-4ea5-9cc2-468ff0ad8274/edit?viewport_loc=3089%2C-363%2C2989%2C2187%2C0_0&invitationId=inv_42059c2e-11b5-4321-b419-7e4b525f7677

UML Class



Crows Foot ERD



SQLite Creation

The following statements are used to generate the tables in the database, and is generated directly from the ERD in LucidChart.

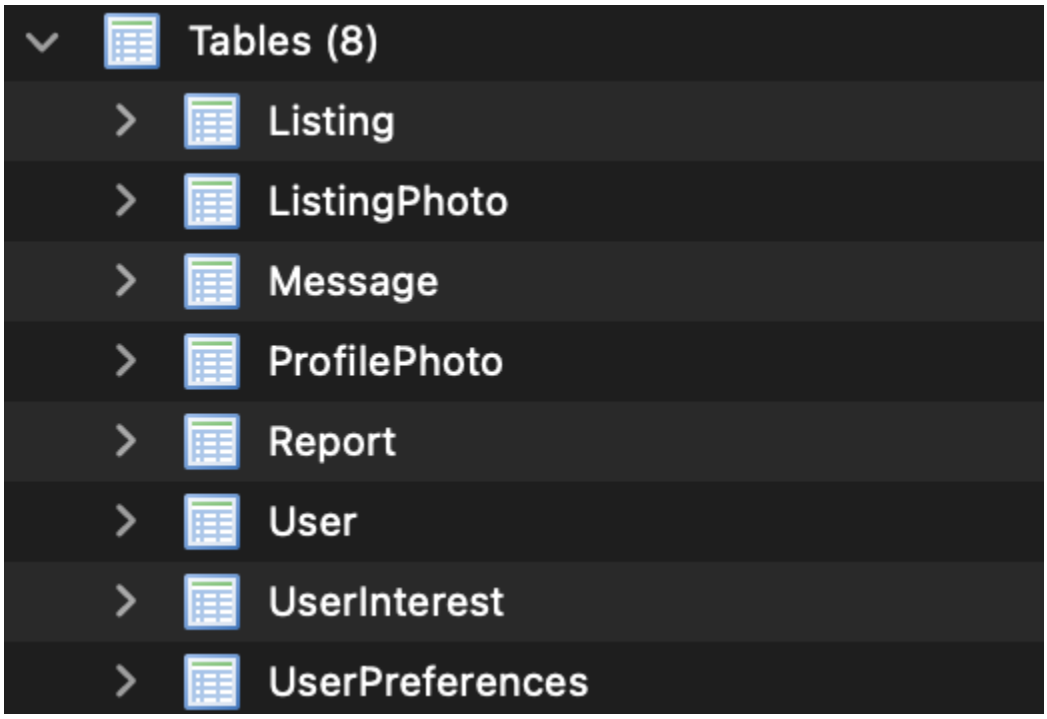
```
CREATE TABLE `User` (
  `user_id` Text,
  `name` Text,
  `email` Text,
  `password_hash` Blob,
  `age` Integer,
  `gender` Text,
  `occupation` Text,
  `bio` Text,
  `profile_photo` Text,
  `created_at` Integer,
  PRIMARY KEY (`user_id`)
);
```

```sql
CREATE TABLE `UserPreferences` (
  `preference_id` Text,
  `user_id` Text,
  `cleanliness_level` Text,
  `sleep_schedule` Text,
  `pet_friendly` Text,
  `smoking_allowed` Text,
  `noise_tolerance` Text,
  `guests_allowed` Text,
  `work_schedule` Text,
  PRIMARY KEY (`preference_id`),
  FOREIGN KEY (`user_id`) REFERENCES `User`(`user_id`)
);

CREATE TABLE `Listing` (
  `listing_id` Text,
  `user_id` Text,
  `title` Text,
  `description` Text,
  `rent_price` Real,
  `location` Text,
  `city` Text,
  `state` Text,
  `zip_code` Text,
  `available_date` Text,
  `num_rooms` Integer,
  `num_bathrooms` Integer,
  `is_active` Text,
  `created_at` Integer,
  PRIMARY KEY (`listing_id`),
  FOREIGN KEY (`user_id`) REFERENCES `User`(`user_id`)
);

CREATE TABLE `ListingPhoto` (
  `photo_id` Text,
  `listing_id` Text,
  `data` Blob,
  PRIMARY KEY (`photo_id`),
  FOREIGN KEY (`listing_id`) REFERENCES `Listing`(`listing_id`)
);

CREATE TABLE `ProfilePhoto` (
  `photo_id` Text,
```

```sql
  `user_id` Text,
  `data` Blob,
  PRIMARY KEY (`photo_id`),
  FOREIGN KEY (`user_id`) REFERENCES `User`(`user_id`)
);

CREATE TABLE `UserInterest` (
  `interest_id` Text,
  `renter_id` Text,
  `listing_id` Text,
  `status` Text,
  `created_at` Integer,
  PRIMARY KEY (`interest_id`),
  FOREIGN KEY (`renter_id`) REFERENCES `User`(`user_id`),
  FOREIGN KEY (`listing_id`) REFERENCES `Listing`(`listing_id`)
);

CREATE TABLE `Message` (
  `message_id` Text,
  `sender_id` Text,
  `receiver_id` Text,
  `listing_id` Text,
  `content` Blob,
  `sent_at` Integer,
  `read` Text,
  PRIMARY KEY (`message_id`),
  FOREIGN KEY (`sender_id`) REFERENCES `User`(`user_id`),
  FOREIGN KEY (`receiver_id`) REFERENCES `User`(`user_id`),
  FOREIGN KEY (`listing_id`) REFERENCES `Listing`(`listing_id`)
);

CREATE TABLE `Report` (
  `report_id` Text,
  `reporter_id` Text,
  `reported_listing_id` Text,
  `reason` Text,
  `created_at` Integer,
  PRIMARY KEY (`report_id`),
  FOREIGN KEY (`reported_user_id`) REFERENCES `User`(`user_id`),
  FOREIGN KEY (`reported_listing_id`) REFERENCES `Listing`(`listing_id`)
```

Backend

The backend for this application will be built in DB Browser for sqlite. After running the queries above it correctly generates all entities.



SQL Queries

Query 1 - Users Who Have a Listing Posted

SQL

```
SELECT Users.name, Listings.description
FROM Users LEFT JOIN Listings
ON Users.user_id = Listings.user_id
```

Sample Output

	name	description
7	Riccardo Sapena	*NULL*
8	Court Plinck	*NULL*
9	Ileana Hutchcraft	*NULL*
10	Gwenora Mournian	Reusable silicone lids for covering food in bowls and storag...
11	Allison Crocken	Handy dispenser for quick access to safety pins.
12	Inna Spir	Heavy-duty chain lock for securing bicycles.

Description

This query returns a list of all User's names left-joined with the Listings description where the user_id of the listing matches the User's ID. If the condition is not met for a specific user, the description will be null, so we can easily see who has posts.

Query 2 - Filtering Listing Reports

SQL

```
SELECT
    Report.reason,
    Listings.listing_id,
    Users.name AS user_name,
    Users.user_id AS user_id,
    COUNT(*) OVER(PARTITION BY Listings.listing_id) AS
total_reports_on_listing
FROM Report
INNER JOIN Listings ON Report.reported_listing_id = Listings.listing_id
INNER JOIN Users ON Users.user_id = Listings.user_id
ORDER BY total_reports_on_listing DESC, Listings.listing_id;
```

Sample Output

	reason	listing_id	user_name	user_id	total_reports_on_listing
1	Counterfeit Item	1	Dolores Elrick	56b6bff3-0703-4768-869f-e21df95ce327	4
2	Scam or Fraud	1	Dolores Elrick	56b6bff3-0703-4768-869f-e21df95ce327	4
3	Spam	1	Dolores Elrick	56b6bff3-0703-4768-869f-e21df95ce327	4
4	Counterfeit Item	1	Dolores Elrick	56b6bff3-0703-4768-869f-e21df95ce327	4
5	Scam or Fraud	2	Magda Clough	bb7fca69-d0cd-4ef3-9909-e3a7c01ac664	4
6	Scam or Fraud	2	Magda Clough	bb7fca69-d0cd-4ef3-9909-e3a7c01ac664	4
7	Inappropriate Content	2	Magda Clough	bb7fca69-d0cd-4ef3-9909-e3a7c01ac664	4
8	Spam	2	Magda Clough	bb7fca69-d0cd-4ef3-9909-e3a7c01ac664	4
9	Inappropriate Content	3	Jo ann Wheal	8837b08d-3352-4ad1-be96-a858880c3175	2
10	Counterfeit Item	3	Jo ann Wheal	8837b08d-3352-4ad1-be96-a858880c3175	2

Description

To start the query, we decide three crucial datapoints; the reason for the report, the data from the reported listing, and the number of times that listing has been reported for any reason. This number can be calculated by counting the length of each group partitioned by the report's listing_id. For convenience, the reported user id is also selected, though it is contained in the listing information. The selection table is created joining reports to listings where the id matches, and joining that to Users on the reported user_id.

Query 3 - Simple Login Functionality

SQL

```
SELECT *
FROM Users
WHERE
      Users.email = "<email>" AND
      Users.password_hash = "<password_hash>"
```

Sample Output

email=vmacclenan26@4shared.com
password_hash=$2a$04$c9VBogI4spicUJDRnBCeiuhXFGWHDbS7KHQQCdUIOjiT91Eqb65FG

	user_id	name	email	password_hash	age
1	fd47c2ed-5cc0-40cd-bb95-e8f1b2b83a8b	Valentine MacClenan	vmacclenan26@4shared.com	$2a$04$c9VBogI4spicUJDRnBCeiuhXFGWHDbS7KHQQC...	58

Description

This query returns all user data where the email and password match. Once the user is authenticated, the app can use the returned relation to send an auth email and fetch data from other tables. If no data is returned, the app knows the email or password is incorrect.

Query 4 - Get Number of Unread Messages Per Sender for a User

SQL

```
SELECT Users.name, COUNT(*) AS unread_messages
FROM Messages
INNER JOIN Users ON Users.user_id = Messages.sender_id
WHERE Messages.receiver_id = '<user_id>'
AND read = "False"
GROUP BY Messages.sender_id, Users.name
```

Sample Output

user_id=1eee0a9f-d67d-4355-bc3f-1a7391439069

	name	unread_messages
1	Aurel Roydon	1
2	Megan Upstone	1
3	Saul Ousley	1
4	Vick Presdee	1
5	Florida Dignum	2
6	Ewell Beinke	1
7	Haleigh Cohrs	1

Description

This query takes in a user id as input and returns the number of messages received by each sender, using the Count aggregate and grouping by the sender_id, only where the message's read status is false. We can use the return relation to send specific notifications or display them in the messages page in the app.

Query 5 - Get Listings for Similar User Preferences

SQL

```
SELECT Listings.*
FROM Listings
INNER JOIN UserPreferences
ON UserPreferences.user_id = Listings.user_id
CROSS JOIN (
    SELECT * FROM UserPreferences WHERE user_id = '<user_id>'
) AS rp
WHERE rp.pet_friendly = UserPreferences.pet_friendly
AND rp.smoking_allowed = UserPreferences.smoking_allowed
AND rp.sleep_schedule = UserPreferences.sleep_schedule
-- Other preferences for matching users
----------------------------------------
-- AND rp.cleanliness_level = UserPreferences.cleanliness_level
```

```
-- AND rp.guests_allowed = UserPreferences.guests_allowed
-- AND rp.work_schedule = UserPreferences.work_schedule
-- AND rp.noise_tolerance = UserPreferences.noise_tolerance
```

Example Output

user_id=1eee0a9f-d67d-4355-bc3f-1a7391439069

	listing_id	user_id	title	description	rent_price	location
1	6	b905ca51-5c0d-4428-822c-1e1fcc828647	Wrap Jumpsuit	Chic wrap jumpsuit that flatters the body and is perfect for...	4784.14	367 Eastlawn P
2	15	a18fcd9f-8595-4b61-8296-898ca4543166	Silicone Stretch Lids	Reusable silicone lids for covering food in bowls and storag...	3120.65	2 Pankratz Par

Description

The query joins the user preferences of the listing owner with the listing itself to a table, and then joins each tuple with the selected users preferences, saving the selected user's preference reference as "rp" for renter preference. Then it filters for matches and returns only the listing properties for the remaining tuples. The preference matching statement is configurable and you can string more checks together to get a better match. This will be used for the preference search function, and an ordering variant can be used to order all listings by preference match.

Query 6 - Get Listings With More Than One Interested Renter

SQL

```
SELECT
      Listings.title,
      Listings.rent_price,
      Listings.city,
      COUNT(*) AS total_interest
FROM UserInterest
INNER JOIN Listings
ON UserInterest.listing_id = Listings.listing_id
GROUP BY UserInterest.listing_id
HAVING COUNT(*) > 1
ORDER BY total_interest DESC
```

Example Output

	title	rent_price	city	total_interest
1	Lentil Vegetable Stew	5013.2	Waco	5
2	Wall-Mounted Organizer	8263.04	Torrance	5
3	Silicone Stretch Lids	3120.65	Colorado Springs	4
4	Cinnamon Sugar Donuts	6040.34	Salinas	4
5	Miso Soup Starter	681.58	Cleveland	2
6	Adjustable Kitchen Broom Stand	4522.12	Palm Bay	2
7	Buffalo Sauce	5725.74	Minneapolis	2
8	Spice Rack	8226.82	Fresno	2
9	Chocolate Chip Cliff Bars	492.46	Abilene	2
10	Himalayan Salt Grinder	8647.81	Albany	2

Description

This query returns all listings that have received interest from more than one renter. It joins UserInterest to Listing on the listing_id, groups by listing, and uses a HAVING clause to filter out listings with only one or zero interested renters. This is useful for listers to see which of their postings are most popular.